Σωροί και ουρές προτεραιότητας

Μια ουρά προτεραιότητας επιστρέφει πάντα το στοιχείο με τη μεγαλύτερη (ή μικρότερη) προτεραιότητα. Χρησιμοποιείται σε χρονοπρογραμματισμό διεργασιών, σε προσομοιώσεις και στον αλγόριθμο Dijkstra.

- Βασικές πράξεις: insert, find-min, extract-min, decrease-key.
- Η πιο συνηθισμένη υλοποίηση είναι ο δυαδικός σωρός (binary heap).

Δυαδικός σωρός

Δυαδικός σωρός ελαχίστου είναι πλήρες δυαδικό δέντρο όπου κάθε κόμβος έχει κλειδί μικρότερο ή ίσο από τα παιδιά του. Άρα το ελάχιστο είναι πάντα στη ρίζα.

- Αποθηκεύεται σε απλό πίνακα: για τον κόμβο στη θέση i, τα παιδιά είναι στις 2i + 1 και 2i + 2 και ο γονέας στη ⌊(i − 1)/2⌋.
- Επειδή το δέντρο είναι πλήρες, το ύψος είναι ⌊log₂ n⌋.

Εισαγωγή και εξαγωγή ελαχίστου

- Εισαγωγή: βάζουμε το στοιχείο στο τέλος του πίνακα και το «ανεβάζουμε» (sift-up) όσο είναι μικρότερο από τον γονέα του. O(log n).
- Εξαγωγή ελαχίστου: παίρνουμε τη ρίζα, βάζουμε στη θέση της το τελευταίο στοιχείο και το «κατεβάζουμε» (sift-down) προς το μικρότερο παιδί. O(log n).

```
import heapq
h = []
heapq.heappush(h, 5); heapq.heappush(h, 2); heapq.heappush(h, 8)
heapq.heappop(h)   # 2
```

Κατασκευή σωρού και heapsort

Από έναν τυχαίο πίνακα φτιάχνουμε σωρό εφαρμόζοντας sift-down από τον τελευταίο εσωτερικό κόμβο προς τη ρίζα. Αν και φαίνεται O(n log n), η ανάλυση δίνει O(n).

- Heapsort: κατασκευή σωρού σε O(n) και μετά n εξαγωγές σε O(log n) η καθεμία, συνολικά O(n log n).
- Ο heapsort είναι in-place (O(1) επιπλέον μνήμη) αλλά δεν είναι σταθερός.

Σύγκριση υλοποιήσεων

Υλοποίηση	insert	find-min	extract-min
Αταξινόμητος πίνακας	O(1)	O(n)	O(n)
Ταξινομημένος πίνακας	O(n)	O(1)	O(1)
Δυαδικός σωρός	O(log n)	O(1)	O(log n)

Ο δυαδικός σωρός είναι ο καλύτερος συμβιβασμός όταν οι εισαγωγές και οι εξαγωγές είναι εξίσου συχνές.

Σύνοψη

- Ουρά προτεραιότητας: βγαίνει πρώτο το στοιχείο με τη μεγαλύτερη προτεραιότητα.
- Δυαδικός σωρός: πλήρες δέντρο σε πίνακα, ελάχιστο στη ρίζα.
- insert και extract-min σε O(log n), κατασκευή σε O(n).
- Heapsort: O(n log n), in-place, όχι σταθερός.

Για την επόμενη φορά: βρείτε τα k μεγαλύτερα στοιχεία ενός πίνακα n στοιχείων σε O(n log k) με σωρό.